Phase I Design Document

Course: COMP 302 - Software Engineering
Group: International group

1. Vision

The vision for **Dungeon KUrawler** is to build a robust, extensible, object-oriented 2D game engine that supports a grid-based dungeon crawler. The primary game mode focuses on exploring randomly generated dungeons, puzzle-solving, combat, and resource management to find a specific hidden "Target Item." Crucially, the system will also feature a secondary "Build Mode" allowing players to design custom levels, emphasizing a modular architecture and high cohesion within the game's mechanics.

2. Functional Requirements

2.1. UML Use Case Diagram (Actors & Use Cases)

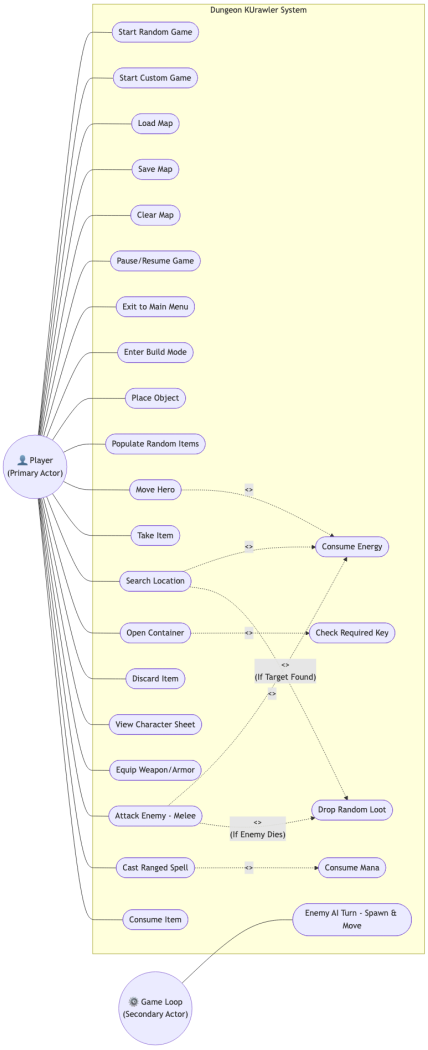

Figure 1: UML Use Case Diagram

Primary Actor: Player
Secondary Actor: Game Loop/Time (System)

- **Session & Level Management:** Start Random Game, Start Custom Game, Load Map, Save Map, Clear Map, Pause/Resume Game, Exit to Main Menu.
- **Editor/Build Mode:** Enter Build Mode, Place Object, Populate Random Items.
- **Hero Actions & Exploration:** Move Hero, Take Item, Search Location, Open Container, Discard Item.
- **Combat & Stats:** View Character Sheet, Equip Weapon, Attack Enemy (Melee), Cast Spell (Ranged), Consume Item.

2.2. Use Case Narratives

(Below are 10 detailed use case narratives representing the most complex interactions in the system).

UC-01: Place Object (Build Mode)

- **Actor:** Player
- **Precondition:** Player is in Build Mode and the map grid is visible.
- **Main Success Scenario:**
  1. Player selects an object (e.g., crate, wall, monster) from the top item palette.
  2. Player drags the object over a specific (x, y) tile on the map.
  3. Player releases the mouse button to drop the object.
  4. System validates that the target tile is empty and passable.
  5. System updates the Map data structure to include the new object at (x, y).
  6. System renders the object on the grid.
- **Alternative Scenarios:**
  - **4a. Target tile is occupied:** System displays a visual warning (e.g., red highlight) and cancels the placement; the dragged object snaps back to the palette.

UC-02: Start Random Game (Target Item Objective)

- **Actor:** Player
- **Precondition:** Player is at the Main Menu.
- **Main Success Scenario:**
  1. Player clicks "Start Random Game".
  2. System generates a random map layout consisting of static objects (walls, columns).
  3. System spawns a fixed number of random items (weapons, potions) on empty floor tiles.
  4. System randomly selects a valuable item (e.g., Diamond, Crystal Orb) to act as the Target Item.
  5. System selects a random container (e.g., chest) or searchable location (e.g., fountain) on the map.
  6. System hides the Target Item inside the selected container/location.
  7. System displays a UI prompt showing the Target Item to the player and asks them to find it.
  8. Player clicks "Acknowledge" and gameplay begins.

UC-03: Move Hero

- **Actor:** Player
- **Precondition:** Game is in Play Mode, Hero is alive.
- **Main Success Scenario:**
  1. Player presses a directional arrow key (e.g., North).
  2. System calculates the target coordinates (x, y+1).
  3. System verifies the target tile is within map bounds and not blocked by a static impassable object (e.g., Wall).
  4. System updates the Hero's (x, y) coordinates.
  5. System deducts the required movement Energy cost from Hero's stats.
  6. System updates the UI to reflect the new position and Energy level.
- **Alternative Scenarios:**

- **5a. Hero lacks sufficient Energy:** System plays an error sound and Hero remains in place.

UC-04: Take Item

- **Actor:** Player
- **Precondition:** Hero is located in the 3x3 grid adjacent to a collectible item.
- **Main Success Scenario:**
  1. Player clicks on the collectible item on the ground.
  2. System displays an action menu for the item.
  3. Player selects "Take [Item Name]".
  4. System checks if the Hero's 2x4 Inventory has at least one empty slot.
  5. System removes the item from the Map grid.
  6. System adds the item to the first available Inventory slot.
  7. System closes the action menu and updates the UI.
- **Alternative Scenarios:**
  - **4a. Inventory is full:** System displays the "Take" option as grayed out with a "[Inventory Full]" tag. The action cannot be clicked.

UC-05: Attack Enemy (Melee)

- **Actor:** Player
- **Precondition:** Hero has a melee weapon equipped and is adjacent to an Enemy.
- **Main Success Scenario:**
  1. Player clicks on an adjacent Enemy (Knight or Sorcerer).
  2. System retrieves Hero's STR stat and equipped Weapon's ATK value to compute Base Damage.
  3. System retrieves Enemy's DEF stat (including armor modifiers).
  4. System calculates Final Damage = Base Damage - Enemy DEF.
  5. System reduces Enemy HP by Final Damage.
  6. System deducts Energy from the Hero.
  7. If Enemy HP <= 0, System removes the Enemy from the Map and rolls for loot drop.

UC-06: Cast Ranged Spell

- **Actor:** Player
- **Precondition:** Hero has a Spell Book in inventory and sufficient Mana.
- **Main Success Scenario:**
  1. Player opens inventory and clicks the Spell Book.
  2. Player selects "READ".
  3. System verifies Hero has enough Mana (e.g., >= 10).
  4. System deducts Mana and instantiates a magical projectile traveling in the direction the Hero is facing.
  5. Projectile travels cell by cell on a timer.
  6. Projectile intersects with an Enemy tile.
  7. System applies magical damage to the Enemy and destroys the projectile.

UC-07: Equip Weapon/Armor

- **Actor:** Player
- **Precondition:** Hero has a weapon/armor item in their Inventory.
- **Main Success Scenario:**
  1. Player clicks an equippable item in their Inventory.
  2. Player selects "EQUIP" from the action menu.
  3. System checks if an item of the same type (Weapon or Armor) is currently equipped.
  4. If yes, System automatically unequips the old item (it remains in inventory, status updated).

5. System marks the new item as “Equipped”.
6. System recalculates Hero’s total ATK or DEF stats and updates the Character Sheet UI.

UC-08: Open Container with Key

- **Actor:** Player
- **Precondition:** Hero is adjacent to a locked Chest and possesses the correct Key.
- **Main Success Scenario:**
  1. Player clicks the locked Chest.
  2. System checks the Hero’s inventory for the specific Key associated with this Chest.
  3. System displays the action “Open Chest with Key”.
  4. Player selects the action.
  5. System removes the Key from the inventory (if it is a single-use key).
  6. System changes Chest state to “Open” and displays its contents in a UI window.
  7. Player can click items in the Chest window to move them to the Hero’s Inventory.
  8. If the Target Item is inside, System triggers the “Game Won” sequence.

UC-09: Search Hidden Location

- **Actor:** Player
- **Precondition:** Hero is adjacent to a searchable static object (e.g., fountain, crack in wall).
- **Main Success Scenario:**
  1. Player clicks the static object.
  2. Player selects “SEARCH” from the action menu.
  3. System checks the object’s hidden inventory.
  4. System deducts Energy for the physical effort of searching.
  5. System finds a hidden item.
  6. System displays message: “You have found a [Item Name].”
  7. System automatically transfers the item to the Hero’s inventory (if space permits).
  8. If the item is the Target Item, System triggers the “Game Won” sequence.

UC-10: Enemy AI Turn (Spawn & Move)

- **Actor:** Game Loop (System)
- **Precondition:** 9 seconds have passed since the last spawn cycle.
- **Main Success Scenario:**
  1. System timer triggers the Spawn Event.
  2. System checks current total enemy count (must be < 5).
  3. System selects a random empty edge tile next to a wall.
  4. System generates an enemy (60% Knight, 30% Sorcerer).
  5. System places the new enemy on the tile.
  6. (Sub-flow Move) For Knights within 5 tiles of the Hero, System calculates pathfinding to the Hero’s tile and updates the Knight’s (x,y) coordinates by 1 cell.

3. Domain Model (Classes)

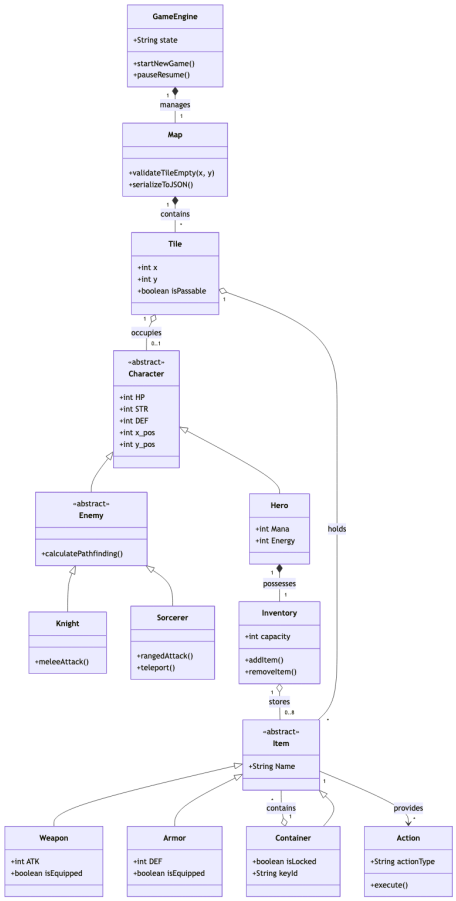

Figure 2: Domain Model Class Diagram

1. **GameEngine:** The root controller managing game states (Build/Play), timers, the render loop, map generation, and objective tracking.
2. **Map:** Represents the current dungeon level. Contains a 2D array/grid of `Tile` objects. Handles pathfinding and bounds checking.
3. **Tile:** A single (x, y) coordinate cell on the Map. Can hold a list of `Entity` objects (e.g., one Static Object, multiple Items, one Character).
4. **Character (Abstract):** Base class for `Hero` and `Enemy`. Contains common attributes: `HP`, `STR`, `DEF`, `x_pos`, `y_pos`.
5. **Hero:** Extends `Character`. Adds specific stats (`Mana`, `Energy`) and holds a reference to an `Inventory` object.
6. **Enemy:** Extends `Character`. Subclassed into `Knight` (melee, moves) and `Sorcerer` (ranged, teleports). Contains AI logic methods.
7. **Item (Abstract):** Represents any interactable object. Possesses an `ActionList` and a `Name`.
8. **Weapon:** Extends `Item`. Contains an `ATK` value and an `isEquipped` boolean flag.
9. **Container:** Extends `Item` (or StaticObject). Contains a list of `Item` objects and a boolean `isLocked`.
10. **Action:** Represents interactions (e.g., TakeAction, EatAction, BreakAction). Executes an `Effect` (e.g., HealthRestoreEffect, DamageEffect, WinGameEffect) upon the target or user.

4. System Sequence Diagrams (SSDs)

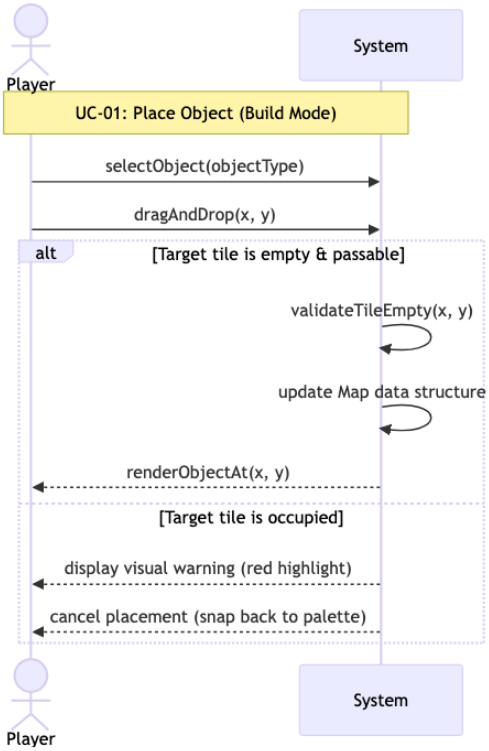

Figure 3: SSD-1: Place Object (Build Mode)

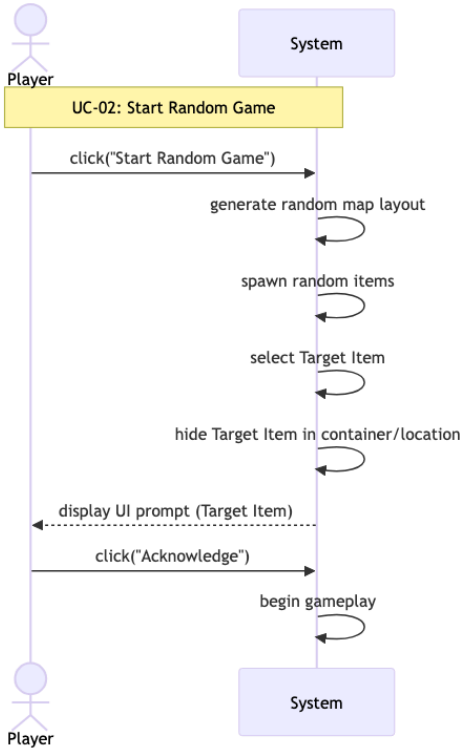

Figure 4: SSD-2: Start Random Game

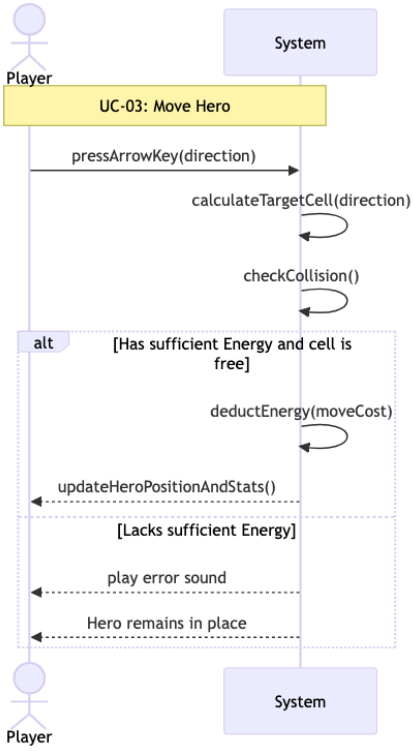

Figure 5: SSD-3: Move Hero

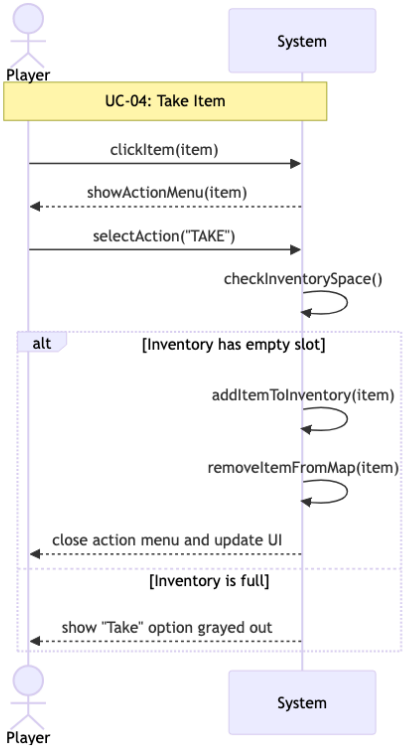

Figure 6: SSD-4: Take Item

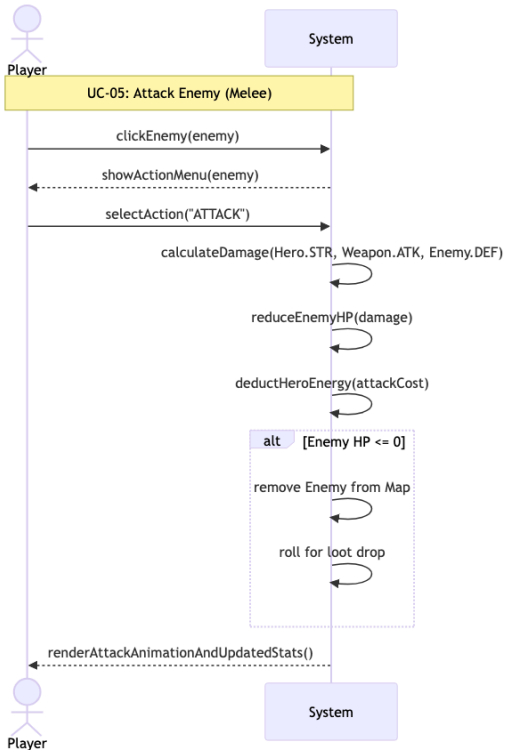

Figure 7: SSD-5: Attack Enemy

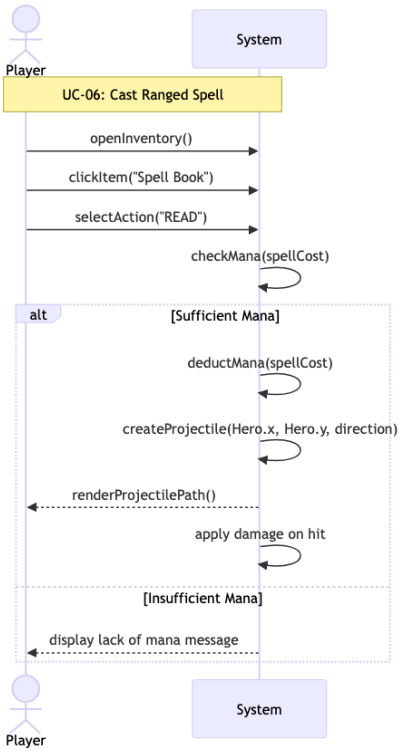

Figure 8: SSD-6: Cast Spell

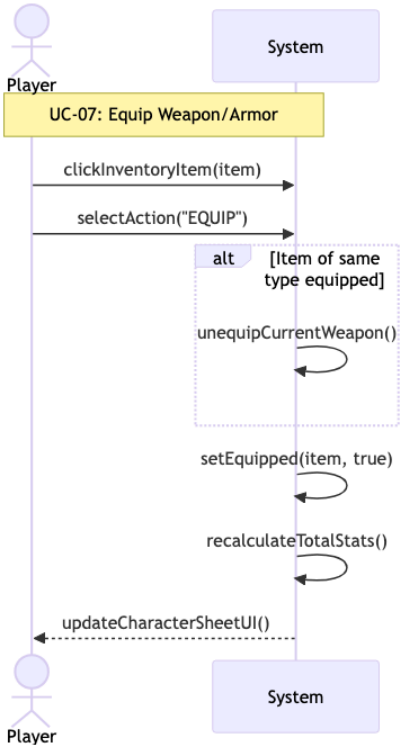

Figure 9: SSD-7: Equip Weapon

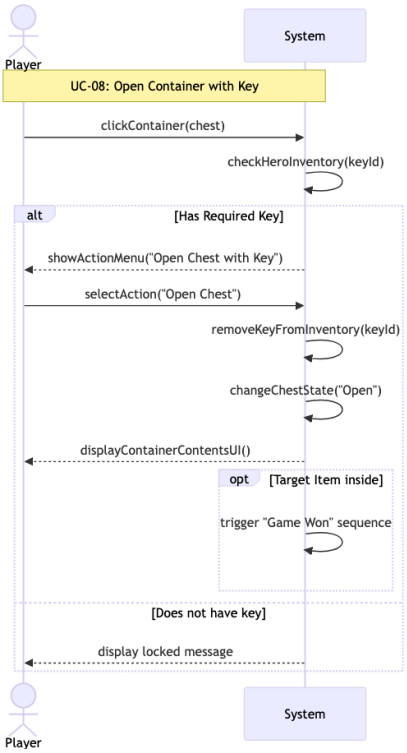

Figure 10: SSD-8: Open Container

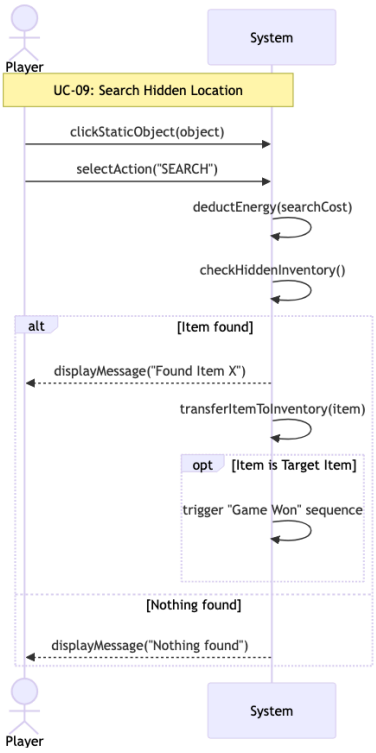

Figure 11: SSD-9: Search Location

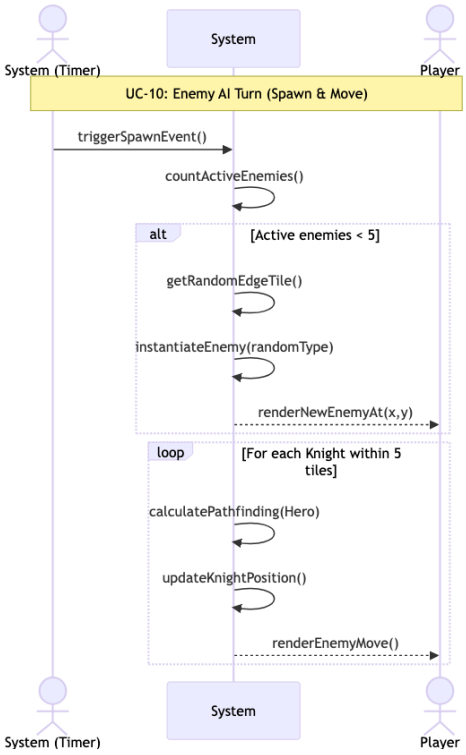

Figure 12: SSD-10: Enemy Turn (Spawn)

5. Operation Contracts

Contract 1: `placeObject(objectType, x, y)`

- **Cross Ref:** UC-01
- **Preconditions:** System is in Build Mode. `x` and `y` are within Map bounds.

- **Postconditions:** An instance of `objectType` is created. `Map.tiles[x][y]` contains the newly created object.

Contract 2: `startRandomGame()`
- **Cross Ref:** UC-02
- **Preconditions:** Player is at the Main Menu.
- **Postconditions:** A new `Map` is generated with random static objects and items. A single `Item` is designated as the Target Item and placed inside a `Container` or searchable location. Game state transitions to Play Mode.

Contract 3: `moveHero(direction)`
- **Cross Ref:** UC-03
- **Preconditions:** Game is in Play Mode. Hero has Energy > 0. Target tile is passable.
- **Postconditions:** Hero's `x_pos` or `y_pos` was updated. Hero's `Energy` was decreased by movement cost.

Contract 4: `takeItem(item)`
- **Cross Ref:** UC-04
- **Preconditions:** `item` is adjacent to Hero. Inventory has < 8 items.
- **Postconditions:** `item` was removed from `Map`. `item` was added to `Hero.Inventory`.

Contract 5: `attackEnemy(enemy)`
- **Cross Ref:** UC-05
- **Preconditions:** Hero is adjacent to `enemy`. Hero has enough Energy to attack.
- **Postconditions:** `enemy.HP` was reduced by computed damage. `Hero.Energy` was reduced. If `enemy.HP <= 0`, `enemy` was removed from `Map`.

Contract 6: `castSpell(spellItem)`
- **Cross Ref:** UC-06
- **Preconditions:** `Hero.Mana` >= `spellItem.manaCost`.
- **Postconditions:** `Hero.Mana` was reduced by `spellItem.manaCost`. A `Projectile` object was instantiated on the Map.

Contract 7: `equipItem(item)`
- **Cross Ref:** UC-07
- **Preconditions:** `item` is in `Hero.Inventory`. `item` is of type Weapon or Armor.
- **Postconditions:** Any previously equipped item of the same type has `isEquipped` set to false. Passed `item` has `isEquipped` set to true. Hero's cumulative stats are updated.

Contract 8: `openContainer(container)`
- **Cross Ref:** UC-08
- **Preconditions:** Hero is adjacent to `container`. If `container.isLocked` is true, Hero has the required Key.
- **Postconditions:** `container.isLocked` is set to false. Container's UI state changes to open, making inner items accessible. If Target Item is revealed, game win condition triggers.

Contract 9: `searchLocation(location)`
- **Cross Ref:** UC-09
- **Preconditions:** Hero is adjacent to `location`. Hero has enough Energy.
- **Postconditions:** `Hero.Energy` was reduced. Random probability check was executed; if successful, a new `Item` was added to `Hero.Inventory`. If Target Item is found, game win condition triggers.

Contract 10: `spawnEnemy()`

- **Cross Ref:** UC-10
- **Preconditions:** Active enemy count is < 5.
- **Postconditions:** A new `Enemy` object (Knight or Sorcerer) was created and added to a random edge `Tile` on the Map.

6. Inception Phase Artifacts

6.1. Supplementary Specification (Non-Functional Requirements)

- **Graphics Quality:** The game must use a uniform pixel size across all sprites and layers. Aspect ratios must remain 1:1 without distortion (no uneven scaling).
- **Audio Feedback:** Interactive elements (buttons, items, combat actions) must produce an audible sound upon interaction.
- **Accessibility & UX:** UI text must maintain high contrast against backgrounds. The game must support keyboard shortcuts alongside mouse clicks (e.g., "I" for Inventory, arrows for movement).
- **Technology Constraints:** Development is restricted to base Java libraries (JavaFX, Swing, or libGDX). No 3rd party wrappers or game engines like Unity are allowed.
- **Persistence:** Level saving and loading must be implemented strictly via JSON serialization.

6.2. Glossary

- **Action List:** A dynamically generated menu of interactions (e.g., Take, Eat, Read) available when a player clicks an object.
- **Build Mode:** The state of the game engine that pauses gameplay mechanics and allows the user to drag and drop tiles to design a map.
- **Effect:** An object-oriented design pattern used to decouple an Action (what the player clicked) from the actual state change (e.g., healing, damage, stat buff).
- **Modal Window:** A user interface window (like a popup menu or chest contents) that halts background interaction until the window is closed or resolved.
- **Target Item:** A randomly designated objective item required to win the game in the random map mode.
- **Passable/Impassable:** A boolean state of a map tile determining whether characters (Hero, Enemies) can walk through it.